

SISTEMA DE BANCO DE ALIMENTOS COMUNITÁRIO

Relatório geral de tecnologias utilizadas

Relatório técnico elaborado conforme formatação acadêmica baseada nas normas ABNT.

Brasil

2026

1 INTRODUÇÃO

O presente relatório descreve, de forma geral, as tecnologias empregadas no Sistema de Banco de Alimentos Comunitário. A aplicação tem como finalidade apoiar o cadastro de usuários, o controle de doadores, o registro de alimentos recebidos e o acompanhamento do estoque disponível, distribuído ou vencido. Trata-se de um sistema web acadêmico, executado em ambiente Node.js, com persistência local em banco SQLite e renderização de páginas por templates EJS.

A análise foi realizada a partir dos arquivos do projeto, especialmente package.json, server.js, models, routes, controllers, middlewares, views e public. Esses arquivos mostram uma arquitetura organizada em camadas, na qual as rotas recebem as requisições, os controladores executam as regras de negócio, os modelos representam as tabelas do banco de dados e as views apresentam as telas ao usuário.

2 OBJETIVO DO SISTEMA

O sistema busca centralizar informações essenciais para a rotina de um banco de alimentos comunitário. Entre suas funções principais estão autenticação de usuários, cadastro e manutenção de doadores, cadastro de alimentos, consulta de itens por status, visualização de detalhes, edição e exclusão de registros. O painel inicial também apresenta indicadores como total de doadores, alimentos disponíveis, itens distribuídos, usuários cadastrados e alimentos próximos ao vencimento.

3 ARQUITETURA GERAL

- Camada de servidor: server.js inicializa o Express, middlewares, rotas, views e conexão com banco.
- Camada de rotas: authRoutes, doadorRoutes e alimentoRoutes separam os endpoints públicos e protegidos.
- Camada de controle: authController, doadorController e alimentoController concentram validações e operações CRUD.
- Camada de dados: Sequelize define os modelos User, Doador e Alimento e seus relacionamentos.
- Camada visual: arquivos EJS em views e ativos CSS/JavaScript em public compõem a interface do usuário.

4 TECNOLOGIAS UTILIZADAS

As tecnologias identificadas no sistema compõem uma aplicação web monolítica, com backend e renderização de telas no mesmo servidor. A tabela a seguir resume os principais recursos utilizados e sua função dentro do projeto.

Tecnologia	Versão identificada	Uso no sistema
Node.js	Ambiente de execução	Executa o servidor JavaScript e os scripts npm.
Express	4.22.2	Framework HTTP responsável por rotas, middlewares e respostas.
EJS	3.1.10	Motor de templates usado nas páginas de login, dashboard, doadores e alimentos.
Sequelize	3.30.0	ORM utilizado para mapear modelos e consultar o SQLite.
SQLite3	6.0.1	Banco de dados relacional local, salvo em database.sqlite.
jsonwebtoken	9.0.3	Geração e validação de tokens JWT para autenticação.
bcryptjs	2.4.3	Criptografia de senhas antes do armazenamento.
cookie-parser	1.4.7	Leitura do token salvo em cookie HTTP.
dotenv	16.6.1	Carregamento de variáveis de ambiente, como porta e segredo JWT.
Nodemon	3.1.14	Reinicialização automática do servidor em desenvolvimento.
HTML, CSS e JavaScript	Arquivos do projeto	Construção da interface, estilos e comportamentos no navegador.

5 MÓDULOS E FUNCIONALIDADES

O módulo de autenticação permite cadastro, login e logout. Durante o cadastro, o sistema verifica campos obrigatórios, impede e-mails duplicados e armazena a senha com hash bcrypt. No login, a senha é comparada com o hash salvo e, se válida, um token JWT é emitido e armazenado em cookie httpOnly.

O módulo de doadores realiza operações de listagem, criação, consulta, edição e exclusão. Os registros possuem nome obrigatório e campos complementares como telefone, e-mail, endereço e responsável. O módulo de alimentos também executa CRUD completo, incluindo nome, quantidade, unidade, data de validade, status e vínculo com o doador.

6 BANCO DE DADOS E SEGURANÇA

A persistência é feita em SQLite, configurado pelo arquivo `config/database.js`. O Sequelize sincroniza os modelos e define relacionamentos em `models/index.js`: um doador pode possuir vários alimentos, um alimento pertence a um doador, um usuário pode possuir vários alimentos e um alimento pode pertencer a um usuário. As principais entidades são User, Doador e Alimento.

Entidade	Campos principais	Finalidade
User	id, name, email, password, role	Controlar usuários, perfis de admin e voluntário e autenticação.
Doador	id, nome, telefone, email, endereço, responsável	Registrar pessoas ou instituições que realizam doações.
Alimento	id, nome, quantidade, unidade, validade, status	Controlar estoque, distribuição e vencimento dos alimentos.

Em segurança, o projeto utiliza JWT para sessão autenticada, cookie `httpOnly` para reduzir exposição do token no navegador e middleware de autorização para proteger rotas privadas. Há ainda controle de perfil administrativo na rota de usuários e em rotas restritas. As senhas não são armazenadas em texto puro, pois passam por hash com `bcryptjs` antes da gravação no banco.

7 CONSIDERAÇÕES FINAIS

O conjunto tecnológico escolhido é adequado a um sistema acadêmico de pequeno e médio porte, pois reduz a complexidade de infraestrutura e permite entregar uma aplicação funcional com autenticação, interface web, banco local e organização por camadas. A combinação de Express, EJS, Sequelize e SQLite favorece manutenção simples e rápida compreensão por equipes em formação.

Como evolução, recomenda-se padronizar testes automatizados, revisar a versão do Sequelize em relação à documentação atual, fortalecer validações de entrada, ampliar tratamento de erros e preparar estratégia de backup ou migração caso o sistema seja utilizado em ambiente real.

REFERÊNCIAS

ARQUIVO `package.json` do projeto. Dependências e scripts da aplicação.

ARQUIVO `server.js` do projeto. Configuração do servidor, rotas e dashboard.

DIRETÓRIOS `models`, `controllers`, `routes`, `middlewares`, `views` e `public`. Estrutura MVC e interface do sistema.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. Diretrizes gerais de apresentação de trabalhos acadêmicos.